JavaScript Essentials for Test Automation

COURSE SPECIFICATIONS

• Track: Playwright Automation Testing (JavaScript) • Level: Beginner
• Focus: JavaScript Essentials for Automation QA • Prerequisites: Day 1 Automation Fundamentals
• Runtime: Node.js (v18+) & Modern ES6+ JavaScript • Modules: ESM import/export syntax

Learn the essential JavaScript you need for Playwright test automation: variables, conditions, loops, functions, arrays, objects, and async/await—explained step by step with practical QA examples.

Day 2 Learning Objectives

'Understand JavaScript syntax, variables (let vs const), and core data types (strings, numbers, booleans, null, undefined).
'Use if/else conditions and strict equality (===) to verify test outcomes accurately.
'Write clean for and for...of loops to repeat test steps across multiple data inputs safely.
'Build reusable helper functions and arrow functions to keep your test code clean and maintainable.
'Organize test data with arrays and objects to prepare for data-driven testing.
'Master asynchronous programming with Promises, async/await, and error handling with try...catch.

JavaScript is the programming language that powers the web. Every time you click a button, open a popup modal, or submit a form on a website, JavaScript is running behind the scenes.

Why JavaScript Matters for Automation
When you write tests, your instructions are written in standard JavaScript. Your code tells the browser what to do—such as navigating to a URL, typing into an email field, clicking a submit button, and verifying that the correct dashboard loads.

How JavaScript Compares to HTML and CSS
Think of a web page like a house:
• **HTML (Structure): The walls and doors. HTML defines elements like headings, text inputs, buttons, and tables.**
• **CSS (Style): The paint, furniture, and decorations. CSS controls colors, fonts, spacing, and layouts.**
• **JavaScript (Action): The electricity, plumbing, and smart locks. JavaScript brings the page to life. In test automation, JavaScript drives user actions and checks test results.**

Basic Rules of JavaScript Syntax
• **Statements: A statement is a single instruction telling the computer to do something. In JavaScript, each statement usually ends with a semicolon (`;`).**
• **Expressions: An expression is any piece of code that produces a value. For example, `5 + 3` evaluates to `8`.**
• **Comments: Use comments to leave helpful notes for humans. JavaScript completely ignores them:**
** - Single-line comment: `// This is a note`**
** - Multi-line comment: `/* This is a longer explanation */`**
• **Case Sensitivity: JavaScript cares about uppercase and lowercase letters. The variables `userEmail`, `UserEmail`, and `USEREMAIL` are treated as three completely different variables.**

The 5 Core Data Types for Testers
Every piece of data you handle in automated testing falls into one of these basic types:
1. **String (Text): Written inside quotes (`"text"`, `'text'`, or backticks ``text``). Used for URLs, usernames, page titles, and button labels. Example: `"RootAccess"`**
2. **Number: Numbers without quotes (both whole numbers and decimals). Used for counters, prices, and HTTP status codes. Example: `200`, `49.99`**
3. **Boolean (True or False): Only two possible values: `true` or `false`. Used for flags and checks. Example: `const isTestPassed = true;`**
4. **Null: Represents an intentional empty value. Example: `let errorMessage = null;` (meaning no error has occurred yet).**
5. **Undefined: Means a variable has been declared, but no value has been assigned to it yet.**

How to Declare Variables: const vs let vs var
A variable is a labeled container that stores data for later use. Modern JavaScript gives us two main keywords to create variables:
• **const (Constant — Recommended by Default):**
** Use `const` for any value that should never be reassigned. In test automation, 95% of your variables should be `const` (such as URLs, element locators, and expected test results). If you accidentally try to change a `const` variable, JavaScript immediately gives you an error, preventing accidental bugs.**
• **let (Reassignable):**
** Use `let` only when you know the value needs to change over time, such as a loop counter or a retry counter**

(`let attempts = 0;`).
• **Why You Should Avoid var:**
Before 2015, JavaScript used `var`. However, `var` has confusing scoping rules that can cause subtle bugs. Modern QA engineering codebases avoid `var` entirely. Always use `const` by default, and use `let` when you need to change a value.

Declaring Variables and Printing Values (Declaring test variables with const and let, then logging them to the console.)

```
const applicationName = "RootAccess";
let retryCount = 2;
const isTestPassed = true;

console.log(applicationName);
console.log(retryCount);
console.log(isTestPassed);
```

Expected Output: RootAccess
2
true

Every automated test boils down to one fundamental question:
Does what actually happened match what we expected to happen?

For example:
- Did the user get redirected to the dashboard after logging in?
- Did the API return an HTTP status code of 200?
- Is the error message displayed on the screen?

To check these questions in JavaScript, we use **conditions** and **comparisons**.

Conditional Statements: if, else if, else
Conditional statements allow your code to take different actions depending on whether a condition is true or false:

```javascript
if (actualStatus === 200) {
  console.log("Test Passed: Server returned success status.");
} else if (actualStatus === 404) {
  console.log("Test Failed: Page not found.");
} else {
  console.log("Test Failed: Unexpected status code received.");
}
```

Strict Equality (===) vs Loose Equality (==)
This is one of the most common interview questions for QA automation roles:
• **Strict Equality (`===`) — The Right Way:**
Compares both the **value and the **data type** without any automatic conversion.**
- `200 === 200` -> `true` (both are numbers with value 200)
- `200 === "200"` -> `false` (one is a number, the other is a string)
• **Loose Equality (`==`) — The Risky Way:**
Tries to automatically convert the types before comparing them.
- `200 == "200"` -> `true` (JavaScript automatically converts the string to a number)
• **Why Testers Must Always Use Strict Equality (`===`):**
If an API response sends back the status code as text (`"200"`) instead of a number (`200`), that is a real backend bug! If you use `==`, your test will pass and hide the bug. Always use `===` (strict equal) and `!==` (strict not equal) in your automated test scripts.

Common Comparison Operators
• Greater than / Less than: `>`, `<`
• Greater than or equal / Less than or equal: `>=`, `<=`
• Strict inequality: `!==` (checks that value or type is different)

Logical Operators
• **AND (`&&`): Both sides must be true.**
Example: `statusCode === 200 && responseTimeMs < 500`
• **OR (`||`): At least one side must be true.**
Example: `role === "admin" || role === "instructor"`
• **NOT (`!`): Inverts a boolean value (turns true to false, and false to true).**
Example: `!isButtonDisabled` (meaning the button is enabled).

Truthy and Falsy Values

In JavaScript, any value can be evaluated inside an `if` condition.
There are only **6 Falsy values in all of JavaScript (they count as false):**
1. `false`
2. `0`
3. `""` (empty string)
4. `null`
5. `undefined`
6. `NaN` (Not-a-Number)

Every other value in JavaScript is **Truthy** (evaluates to true). This includes numbers like `1`, non-empty strings like `"hello"`, empty arrays `[]`, and empty objects `{}`.

Verifying an HTTP Status Code (Comparing an actual HTTP response status code to the expected value using strict equality.)

```
const actualStatus = 200;
const expectedStatus = 200;

if (actualStatus === expectedStatus) {
  console.log("Status matches");
} else {
  console.log("Status does not match");
}
```

Expected Output: Status matches

In test automation, you often need to repeat the same check across a list of items.

For example:
- Verifying that every product card on an e-commerce page displays an "Add to Cart" button.
- Running a search test with several different keywords ("laptop", "headphones", "keyboard").
- Checking that all options in a country dropdown menu are present.

Instead of writing the same code 10 times, you use a **loop** to run the check automatically for each item.

The for...of Loop (The Easiest & Most Readable)

The modern `for...of` loop is the cleanest way to iterate through an array of items:

```javascript
const searchTerms = ["laptop", "headphones", "keyboard"];

for (const term of searchTerms) {
  console.log(`Testing search term: ${term}`);
}
```

In plain English, this reads: *"For each term inside the searchTerms list, run this block of code."* On each cycle, the variable `term` receives the next item in the list.

Traditional for Loop (When You Need the Item Number)

If you need to know the index position of each item (for example, to print "Step 1", "Step 2", "Step 3"), use the standard index-based `for` loop:

```javascript
const testSteps = ["Open URL", "Enter Username", "Click Submit"];

for (let i = 0; i < testSteps.length; i++) {
  console.log(`Step ${i + 1}: ${testSteps[i]}`);
}
```

Key points to remember:
• Arrays start counting from index `0` (so `testSteps[0]` is `"Open URL"`).
• We declare `let i = 0` because the counter `i` increments (`i++`) on every turn.
• `testSteps.length` gives the total number of items in the array.

While Loops and Infinite Loop Warnings

A `while` loop keeps running as long as its condition remains true:

```javascript
let retryAttempts = 0;
while (retryAttempts < 3) {
  console.log(`Attempt ${retryAttempts + 1}`);
  retryAttempts++; // Don't forget this!
}
```

```

**Avoiding Common Loop Pitfalls in Automation**
**• **Infinite Loops:** If your loop's condition never becomes false (for example, if you forget `retryAttempts++`), your test runner will freeze and hang forever until the build server times out.**
**• **Overloading a Single Test:** Putting 50 test scenarios inside a single test loop means that if scenario #2 fails, scenarios #3 through #50 will never run! On Day 9, we will learn how to use Playwright's parameterized tests so that each data item runs as an independent, isolated test case.**

**Iterating Through Search Terms (Looping through an array of search queries and printing each term.)**

```
const searchTerms = ["laptop", "headphones", "keyboard"];

for (const term of searchTerms) {
  console.log(`Testing search term: ${term}`);
}
```

*Expected Output: Testing search term: laptop*
*Testing search term: headphones*
*Testing search term: keyboard*

When writing automated tests, you will often perform the exact same tasks over and over:
- Logging in with an admin or student account
- Generating a unique, random email address
- Formatting a date or currency value
- Waiting for a modal dialog to close

If you copy-paste the same 10 lines of code across 30 different test files, what happens when the login button changes? You have to manually edit all 30 files!

Instead, you write a **function** once, and reuse it everywhere.

**What Is a Function?**
**A function is a reusable block of code designed to do a specific job. You define the function once, and then call (invoke) it whenever you need it.**

**Standard Function Declarations**
**A standard function uses the `function` keyword, has a name, accepts optional **parameters**, and returns a value using `return`:**

```javascript
function createTestMessage(testName) {
  return `Running test: ${testName}`;
}

console.log(createTestMessage("Login"));
```

Parameters vs Arguments (Interview Question)
This is a standard question in QA interviews:
• **Parameter: The placeholder variable name listed in the function definition (like `testName` in `function createTestMessage(testName)`).**
• **Argument: The actual, real value you supply when you call the function (like `"Login"` in `createTestMessage("Login")`).**

Return Values
The `return` statement sends a value back to whoever called the function and immediately ends the function. If a function has no `return` statement, it returns `undefined` by default.

Arrow Functions (ES6)
Arrow functions provide a clean, modern shorthand for writing functions using the `=>` ("fat arrow") syntax:

```javascript
// Multi-line arrow function
const createTestMessage = (testName) => {
  return `Running test: ${testName}`;
};

// Single-line arrow function with automatic (implicit) return
const add = (first, second) => first + second;

console.log(add(2, 3)); // Outputs: 5
```
```

In Playwright, you will write arrow functions constantly, especially inside test definitions:
`test('User can check out', async ({ page }) => { ... });`

Variable Scope: Where Can Variables Be Accessed?

• **Block Scope: Variables declared with `const` and `let` inside curly braces `{ ... }` (such as inside an `if` statement or loop) only exist inside those braces.**
• **Function Scope: Variables declared inside a function cannot be accessed from outside that function.**
• **Global Scope: Variables declared outside any function or block can be accessed anywhere in the file.**

Named Function Declaration (Defining and calling a helper function that generates formatted test messages.)

```
function createTestMessage(testName) {
  return `Running test: ${testName}`;
}

console.log(createTestMessage("Login"));
```

Expected Output: Running test: Login

Concise Arrow Function (Using an arrow function with implicit return to calculate a sum.)

```
const add = (first, second) => first + second;

console.log(add(2, 3));
```

Expected Output: 5

Automated tests need structured data to work with. You need to model realistic entities such as user accounts, shopping cart items, address forms, and API payloads.

In JavaScript, **arrays** and **objects** are the two primary tools for organizing test data.

Arrays: Ordered Lists of Items

An array is an ordered list of values enclosed in square brackets `[ ... ]`:

```javascript
const browserEngines = ["chromium", "firefox", "webkit"];

console.log(browserEngines[0]); // "chromium" (first item)
console.log(browserEngines.length); // 3 (total items)

// Adding an item to the end of the array
browserEngines.push("edge");
```

Key array rules:
• **Zero-Based Indexing:** The first item is at index `0`, the second item is at index `1`, and so on.
• **.length property:** Tells you how many items are in the array.
• **.push(item):** Adds a new item to the end of the array.

Objects: Structured Key-Value Pairs

An object represents a single entity with named properties. It is written using curly braces `{ ... }` containing comma-separated `key: value` pairs:

```javascript
const loginData = {
  username: "tester@example.com",
  password: "ExamplePassword123",
  expectedResult: "success"
};
```

Accessing object properties:
• **Dot Notation (Recommended):** `loginData.username` -> `"tester@example.com"`
• **Bracket Notation:** `loginData["password"]` -> `"ExamplePassword123"` (useful when property names contain hyphens or are stored in variables).

Security Best Practice: Always use fictional dummy credentials in your test exercises and code repositories. Never put real production passwords in test files.

Arrays of Objects: The Foundation of Data-Driven Testing

In professional QA, you test features against multiple user types and scenarios (valid logins, wrong passwords, locked accounts, and empty fields).

You represent these scenarios as an array of objects:

```javascript
const loginScenarios = [
  { username: "valid@example.com", expectedResult: "success" },
  { username: "invalid@example.com", expectedResult: "failure" }
];

for (const scenario of loginScenarios) {
  console.log(scenario.username, scenario.expectedResult);
}
```

This pattern is the foundation for **Day 9 (Data-Driven Testing)**, where we load external test datasets from JSON files and automatically generate tests for each scenario!

Single Test Data Object (Declaring a login credentials object and accessing properties with dot notation.)

```
const loginData = {
  username: "tester@example.com",
  password: "ExamplePassword123",
  expectedResult: "success"
};

console.log(loginData.username);
console.log(loginData.expectedResult);
```

Expected Output: tester@example.com
success

Array of Test Scenario Objects (Looping through multiple test scenarios and logging their parameters.)

```
const loginScenarios = [
  { username: "valid@example.com", expectedResult: "success" },
  { username: "invalid@example.com", expectedResult: "failure" }
];

for (const scenario of loginScenarios) {
  console.log(scenario.username, scenario.expectedResult);
}
```

Expected Output: valid@example.com success
invalid@example.com failure

Browser automation is naturally asynchronous. When your test script clicks a button, the browser sends a network request to a server, waits for the response, downloads data, and draws elements onto the screen.

None of this happens in zero milliseconds!

Synchronous vs Asynchronous Execution

• **Synchronous (Blocking): Code runs strictly line by line. Each line must completely finish before the next line can start. If an operation takes 5 seconds, the entire program freezes and waits.**
• **Asynchronous (Non-Blocking): The program starts a task that takes time (such as loading a webpage or fetching API data) and continues running other code without freezing. When the task finishes, JavaScript receives a signal that the result is ready.**

What Is a Promise?

A **Promise is a JavaScript object that represents a future result. Think of it like a restaurant buzzer:**
1. You order food at the counter. The cashier hands you a buzzer. That buzzer is a **Promise.**
2. **Pending: The kitchen is currently preparing your meal.**
3. **Fulfilled (Resolved): The buzzer rings! Your food is ready and handed to you.**
4. **Rejected: The kitchen ran out of ingredients and tells you they cannot make your order.**

In Playwright, almost every browser action—such as `page.goto()`, `locator.click()`, and `locator.fill()`—returns a Promise because the browser needs time to complete the action.

The async and await Superpowers

In modern JavaScript, `async` and `await` make working with Promises simple and readable:
• **async keyword: Placed before a function declaration (`async function runTest() { ... }`). An async function always returns a Promise.**
• **await keyword: Placed before an operation that returns a Promise (`await page.click('button');`). It tells JavaScript:**
"Pause right here. Wait until the browser finishes this action before moving to the next line."
• **Golden Rule: You can only use `await` inside an `async` function.**

```javascript
function getTestResult() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Test data loaded");
    }, 500);
  });
}

async function runExample() {
  try {
    const result = await getTestResult();
    console.log(result);
  } catch (error) {
    console.error("An error occurred:", error);
  }
}

runExample();
```

(Note: setTimeout is used here only as an educational example. In real Playwright tests, you will not use arbitrary sleep timeouts; Playwright's auto-waiting handles timing automatically!)

Handling Errors with try...catch

Asynchronous actions can sometimes fail (for example, if a network connection drops or an element is not found). Wrapping your code in a `try...catch` block lets you handle errors gracefully without crashing the whole test run:

```javascript
try {
  const result = await executeDangerousStep();
} catch (error) {
  console.error("Step failed gracefully:", error.message);
}
```

ES Modules: import and export

In modern JavaScript, you split your code into organized files and share functions between them:

```javascript
// helpers.js
export function formatCurrency(amount) {
  return `$${amount.toFixed(2)}`;
}

// testScript.js
import { formatCurrency } from './helpers.js';
console.log(formatCurrency(49.9)); // "$49.90"
```

Starting on Day 3, every Playwright test file you write will import Playwright fixtures using this clean module syntax!

Asynchronous Promise with async/await (Awaiting a Promise that resolves after a simulated background delay.)

```
function getTestResult() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Test data loaded");
    }, 500);
  });
}

async function runExample() {
  try {
    const result = await getTestResult();
    console.log(result);
  } catch (error) {
    console.error("An error occurred:", error);
  }
}

runExample();
```

Expected Output: Test data loaded

structured test data objects containing test descriptions, user credentials, environmental tags, and execution states.

Task: Build a complete, functional test data utility module in pure JavaScript meeting all 10 deterministic rubric criteria.

Assignment Starter Code (utils/testDataUtility.js)

```
// ============================================================================
// RootAccess Day 2 Practical Assignment: JavaScript Test Data Utility
// ============================================================================

// 1. Declare an array with at least 5 login test scenario objects
const loginScenarios = [
  {
    id: "TC-LOGIN-001",
    username: "student.valid@example.com",
    scenarioType: "positive",
    expectedResult: "dashboard_redirect"
  },
  {
    id: "TC-LOGIN-002",
    username: "wrong.password@example.com",
    scenarioType: "negative",
    expectedResult: "invalid_credentials_error"
  },
  {
    id: "TC-LOGIN-003",
    username: "unregistered.user@example.com",
```

10 Rubric Requirements for Day 2 Assignment:

1. Valid JavaScript syntax (pure JavaScript without syntax errors)
2. Define array of at least 5 test scenario objects with id, title, role, priority, and runAsync properties
3. Declare variables using const and let only — zero occurrences of var
4. Implement strict equality checks (=== and !==) for filtering or status validation
5. Implement a summary calculation function using a loop (for...of or for) or array higher-order method
6. Implement an asynchronous processing function simulating async test preparation using Promise and async/await
7. Include error handling with try...catch around async operations
8. Export utility functions using ES Module export syntax
9. Provide 2-3 paragraph written explanation of how this utility directly applies to Playwright tests
10. Provide clear console output demonstrating summary metrics and processed async results

Q1. What is the difference between let and const in JavaScript?

[Difficulty: Beginner] What Interviewer Evaluates: Variable mutability, scoping, and modern ES6 best practices in automation.

Suggested Answer: Both let and const are block-scoped declarations introduced in ES6. const creates an immutable binding that cannot be reassigned after initialization, making it ideal for locator references, test configs, and expected outcomes. let allows variable reassignment, which is necessary for loop counters and retry accumulators. In modern automation codebases, we default to const to avoid accidental state mutations, using let only when reassignment is strictly required.

Follow-Up Question: "Can properties inside an object declared with const still be modified?"

Q2. What is the difference between == and ===?

[Difficulty: Beginner] What Interviewer Evaluates: Understanding type conversion and preventing false-positive test results.

Suggested Answer: === is strict equality, which compares both the value and the data type without conversion. == is loose equality, which attempts to convert operands to a matching type before comparing (for example, 200 == "200" is true). In test automation, we always use === because receiving a string "200" when expecting a numeric 200 is an API contract bug that loose equality would dangerously hide.

Follow-Up Question: "What are the 6 falsy values in JavaScript?"

Q3. What is the difference between an array and an object?

[Difficulty: Beginner] What Interviewer Evaluates: Data structures for organizing test entities and scenario datasets.

Suggested Answer: An array is an ordered, zero-indexed collection of items (such as [1, 2, 3]), accessed by numerical position (array[0]). An object is an unordered collection of named key-value pairs (such as { username: "user@test.com", role: "admin" }), accessed by property keys using dot notation (user.role) or bracket notation (user["role"]). In automated testing, we frequently use arrays of scenario objects to power data-driven test suites.

Follow-Up Question: "When would you use bracket notation instead of dot notation?"

Q4. What is a function, and why is it useful in test automation?

[Difficulty: Beginner] What Interviewer Evaluates: Code reusability, modularity, and reducing test maintenance overhead.

Suggested Answer: A function is a reusable block of code that performs a specific task when called. In test automation, functions encapsulate repetitive actions—such as logging in, generating random test emails, or formatting currency strings. Centralizing logic in functions keeps test files clean and readable. When an application workflow changes, you only update the helper function in one place instead of editing dozens of test files.

Follow-Up Question: "What does a JavaScript function return if there is no return statement?"

Q5. What is the difference between a parameter and an argument?

[Difficulty: Beginner] What Interviewer Evaluates: Core programming vocabulary and function signature understanding.

Suggested Answer: A parameter is the placeholder variable name defined in the function declaration (for example, username in function login(username)). An argument is the actual value passed to that parameter when the function is called (for example, "alex@rootaccess.org" in login("alex@rootaccess.org")).

Follow-Up Question: "Can you specify default values for function parameters in JavaScript?"

Q6. What is an arrow function?

[Difficulty: Beginner] What Interviewer Evaluates: ES6 syntax familiarity and Playwright test block conventions.

Suggested Answer: An arrow function is a concise syntax for defining functions introduced in ES6, using the => operator (for example: const sum = (a, b) => a + b;). Arrow functions do not bind their own this context and support implicit return on single-line expressions. In Playwright, arrow functions are the standard format for test blocks: test("User can login", async ({ page }) => { ... }).

Follow-Up Question: "How does the "this" keyword behave differently in arrow functions compared to standard functions?"

Q7. What is a Promise in JavaScript?

[Difficulty: Beginner] What Interviewer Evaluates: Asynchronous architecture foundations required for modern browser testing.

Suggested Answer: A Promise is an object representing the future completion or failure of an asynchronous operation and its resulting value. A Promise exists in one of three states: pending (in progress), fulfilled (resolved with a result), or rejected (failed with an error). Almost all Playwright browser interactions—such as page.goto(), locator.click(), and page.textContent()—return Promises.

Follow-Up Question: "How did developers handle asynchronous operations in JavaScript before Promises existed?"

Q8. What do async and await do in JavaScript?

[Difficulty: Beginner] What Interviewer Evaluates: Writing clean, readable asynchronous automation scripts.

Suggested Answer: async marks a function as asynchronous, ensuring it always returns a Promise. await can only be used inside an async function; it pauses execution until the Promise resolves, then yields its resolved value. This avoids messy nested callbacks and Promise .then() chains, allowing Playwright tests to read like clear, step-by-step instructions.

Follow-Up Question: "What happens if you use await outside an async function in a non-module file?"

Q9. What is the purpose of try...catch in JavaScript?

[Difficulty: Beginner] What Interviewer Evaluates: Error handling, test diagnostics, and graceful recovery.

Suggested Answer: try...catch is a control flow structure that handles runtime errors without crashing the entire program. Code inside the try block runs normally; if an error occurs, control moves to the catch block where the error can be inspected, logged, or recovered from. In automation, we use try...catch for optional steps (like dismissing promotional popups) or capturing diagnostic screenshots before re-throwing.

Follow-Up Question: "What is the purpose of the optional finally block in a try...catch statement?"

Q10. Why is asynchronous JavaScript important in Playwright?

[Difficulty: Beginner] What Interviewer Evaluates: Practical connection between JavaScript runtime and browser automation mechanics.

Suggested Answer: Browser automation involves network requests, browser rendering pipelines, and server responses, none of which happen instantly. Playwright commands communicate over an asynchronous WebSocket connection to browser engines (Chromium, Firefox, WebKit). Because Playwright APIs return Promises, using async/await ensures your test pauses at each step until the browser actually finishes loading the page, clicking the button, or evaluating an assertion.

Follow-Up Question: "Why is a hardcoded sleep or setTimeout considered an anti-pattern in Playwright?"

1. **Which JavaScript keyword should you use by default when declaring variables that will not be reassigned?**
Key Concept: const should be your default choice in modern JavaScript to ensure variables cannot be accidentally reassigned.

2. **What is the result of the expression: 404 === "404"?**
Key Concept: Strict equality (===) checks both value and type without coercion. 404 is a number and "404" is a string, so the result is false.

3. **Which of the following is considered a "falsy" value in JavaScript?**
Key Concept: 0 is one of JavaScript's 6 falsy values. "false" is a non-empty string and [] and {} are objects, all of which are truthy.

4. **Which loop syntax is the cleanest and most readable for iterating through an array of test strings in modern JavaScript?**
Key Concept: for...of is the recommended modern syntax for iterating over the values of arrays and iterable collections.

5. **In the function definition function login(username, password) { ... }, what are username and password called?**
Key Concept: Variables specified in the function signature are called parameters. The concrete values passed when calling the function are arguments.

6. **What does an arrow function with no curly braces return: const multiply = (a, b) => a * b;?**
Key Concept: Single-line arrow functions without curly braces have an implicit return, returning the evaluated expression directly.

7. **How do you determine the total number of elements in an array named testScenarios?**
Key Concept: The .length property of an array returns the total number of items stored in that array.

8. **Given const testUser = { name: "Alex", role: "admin" };, how do you access the role property using dot notation?**
Key Concept: Dot notation (object.property) is the standard, readable way to access object properties in JavaScript.

9. **What is a Promise in JavaScript?**
Key Concept: A Promise represents an asynchronous value that may not be available immediately but will resolve or reject in the future.

10. **Why do we place the await keyword before Playwright commands like await page.click("button")?**
Key Concept: page.click() returns a Promise. Placing await before it pauses execution until the browser physically executes the click and verifies completion.